Calidad y Preprocesamiento de Datos

MCIC VÍCTOR MANUEL CORZA VARGAS

MC FERNANDO AVITÚA VARELA

Unidad 3 Perfilado de Datos

Objetivo

- Comprender y aplicar técnicas de perfilado de datos para evaluar la calidad de un conjunto de datos, detectando registros duplicados, datos incompletos, valores anómalos y otras dimensiones de calidad.

Introducción al Perfilado de Datos

¿Qué es el perfilado de datos?

- El perfilado de datos es el proceso de examinar y analizar datos para comprender su estructura, contenido y calidad. Es una etapa fundamental en la preparación de datos, permitiendo identificar problemas como valores faltantes, duplicados, errores y anomalías antes del análisis o integración.

Importancia del perfilado de datos

Mejora la calidad de datos: Permite detectar inconsistencias, valores atípicos y errores.

Facilita la limpieza y transformación de datos: Ayuda a definir reglas de preprocesamiento.

Optimiza la integración de datos: Esencial en la consolidación de múltiples fuentes de información.

Permite la toma de decisiones informadas: Datos de calidad mejoran la precisión de modelos analíticos y reportes.

Orden en el DAMA-DMBOK

El **perfilado de datos** (data profiling) es un paso previo a las estrategias de **calidad de datos** (data quality), porque permite descubrir problemas y definir estrategias antes de aplicar mejoras en la calidad de los datos.

Sin un perfilado adecuado, las iniciativas de calidad pueden ser ineficaces o basarse en suposiciones incorrectas.

Orden en el DAMA-DMBOK

Las métricas específicas son obtenidas del perfilado de datos.

Dimensión de Calidad	Métrica	Resultado	Interpretación
Precisión	% de datos incorrectos en nombres o direcciones	12%	12% de los registros tienen errores ortográficos o de formato
Consistencia	% de cuentas con datos incoherentes en distintos sistemas	18%	Datos no coinciden entre la base de clientes y el sistema de facturación
Completitud	% de registros con campos vacíos	9%	9% de los clientes no tienen número de teléfono registrado
Oportunidad	Tiempo promedio para actualizar datos incorrectos	48 horas	Se tarda hasta 2 días en corregir información tras detectar un error

Perfilado de datos

Orden en el DAMA-DMBOK

Data Profiling (Perfilado de datos)

- Se encuentra dentro de la función de Gestión de Calidad de Datos (Data Quality Management).
- Se lleva a cabo como una fase de diagnóstico para analizar la estructura, distribución y relaciones de los datos.
- Ayuda a identificar problemas como valores nulos, inconsistencias, duplicados y anomalías.

Data Quality Assessment & Improvement (Evaluación y mejora de la calidad de datos)

- Basándose en los hallazgos del perfilado de datos, se evalúan las dimensiones de calidad (Precisión, Completitud, Consistencia, Oportunidad, etc.).
- Se implementan estrategias para corregir errores y mejorar la confiabilidad de los datos.

Técnicas principales del perfilado de datos

- Estadísticas descriptivas: Análisis de frecuencias, medias, medianas, desviaciones estándar. (df.describe)
- Detección de duplicados: Identificación de registros repetidos.
- Detección de valores faltantes: Evaluación de la completitud de datos.
- Análisis de valores anómalos: Detección de outliers mediante histogramas y métodos avanzados.
- Medición de calidad en diferentes dimensiones: Evaluación de precisión, completitud, consistencia y oportunidad.

Detección y Medición de Registros Duplicados

Unicidad

¿Qué es la unicidad en los datos?

La unicidad se refiere a que cada registro en un conjunto de datos representa una entidad única y no está repetido. Los registros duplicados pueden generar errores en los análisis, sesgar modelos y causar inconsistencias en bases de datos.

Causas comunes de duplicados

Errores humanos: Datos ingresados más de una vez manualmente.

Diferentes fuentes de datos: Fusiones de bases con redundancias.

Errores en procesos ETL: Integraciones mal diseñadas generan repeticiones.

Técnicas para detectar registros duplicados

Comparación exacta: Identificación de registros idénticos en todos los campos.

Detección de Duplicados Parciales

Comparación difusa (Fuzzy Matching): Uso de métricas como:

Distancia de Levenshtein: Mide la diferencia en caracteres entre dos cadenas.

Coeficiente de Jaccard: Evalúa la similitud entre conjuntos de palabras.

Algoritmo Soundex: Útil para nombres con variaciones ortográficas.

Comparación exacta de duplicados

```
import pandas as pd

data = {'ID': [1, 2, 2, 3, 5, 9], 'Nombre': ['Ana', 'Luis', 'Luis', 'Maria', 'Pablo', 'Pablo']}

df = pd.DataFrame(data)

# Detección de duplicados

duplicados = df.duplicated().sum()

print(f"\nNúmero de registros duplicados: {duplicados}")
```

Detección de Duplicados Parciales

A veces los registros tienen coincidencia en un subconjunto de columnas.

```
import pandas as pd
data = {'ID': [1, 2, 2, 3, 5, 9], 'Nombre': ['Ana', 'Luis', 'Luis', 'Maria', 'Pablo', 'Pablo']}
df = pd.DataFrame(data)
#Detección parcial de duplicados
duplicados_parciales = df.duplicated(subset=['Nombre']).sum()
print(f"\nNúmero de duplicados parciales por el campo nombre: {duplicados_parciales}")
```

Detección de Duplicados Parciales

En otras ocasiones los registros tienen pequeños cambios (espacios extras, diferencias en mayúsculas, errores tipográficos). Para identificar registros similares con pequeñas variaciones en los nombres o valores, podemos utilizar coincidencias difusas.

```
from fuzzywuzzy import fuzz, process

# Comparar dos cadenas de texto

print(fuzz.ratio("Luis", "Luis "))  # Maneja diferencias leves en los nombres

print(fuzz.ratio("Luis", "Luisillo"))  # Evalúa similitud en términos de caracteres
```

¿A qué se le llama comparación difusa?

Los algoritmos como la distancia de Levenshtein, el coeficiente de Jaccard, el algoritmo soundex, entre otros, son conocidos como algoritmos de comparación difusa porque no realizan comparaciones estrictas o exactas, sino que miden el grado de similitud entre dos elementos (como cadenas de texto, conjuntos o vectores) en lugar de verificar una coincidencia binaria (igual o diferente).

Distancia de Levenshtein

La **distancia de Levenshtein** mide el número mínimo de operaciones necesarias para transformar una cadena en otra. Las operaciones permitidas son:

1. **Inserción**: Agregar un carácter.
2. **Eliminación**: Quitar un carácter.
3. **Sustitución**: Cambiar un carácter por otro.

Distancia de Levenshtein

Ejemplo: Convertir "gato" en "pato"

La palabra `"gato"` se puede convertir en `"pato"` en un solo paso:

- Sustituimos la `"g"` por `"p"`.

Distancia de Levenshtein = 1

Distancia de Levenshtein

```
import Levenshtein

# Distancia entre dos palabras
print(Levenshtein.distance("gato", "pato"))  # Salida: 1
print(Levenshtein.distance("silla", "salón"))  # Salida: 3
print(Levenshtein.distance("casa", "caso"))  # Salida: 1
```

Distancia de Levenshtein

Explicación del ejemplo 2:

Queremos transformar `"silla"` en `"salón"`:

1. **Sustituir** `"i"` → `"a"` `resultado: salla`
2. **Sustituir** segunda `"l"` por `"ó"` `resultado: salóa`
3. **Sustituir** `"a"` `por` “`n`” al final → `"salón"`

Distancia de Levenshtein = 3

Distancia de Levenshtein

¿Porqué cuando utilizamos este código, el resultado es 89?

```
from fuzzywuzzy import fuzz, process

# Comparar dos cadenas de texto
print(fuzz.ratio("Luis", "Luis "))  # Maneja diferencias leves en los nombres
```

Distancia de Levenshtein

Similitud

Levenshtein cuenta el número de ediciones (inserciones, eliminaciones o sustituciones) necesarias para convertir una cadena en otra. Siendo el resultado un número entero. Lo que representa un problema, ya que el número de errores que se pueden esperar en una cadena es proporcional a la longitud de esta. Un problema que se puede solucionar normalizando la distancia para obtener el grado de similitud:

$$1 - (lev_{a,b}(|a|, |b|)/max(|a|, |b|))$$

Distancia de Levenshtein

El valor **89** en la comparación `fuzz.ratio("Luis", "Luis ")` usando `fuzzywuzzy` se debe a que la biblioteca usa la **distancia de Levenshtein normalizada** en porcentaje, y la diferencia entre las cadenas es un espacio adicional al final.

- Para transformar `"Luis"` en `"Luis "`, se necesita **una inserción** (añadir el espacio).
- Longitud total de la cadena más larga: **5** (porque `"Luis "` tiene 5 caracteres).

Ratio = (1 – (Distancia de Levenshtein / Longitud de la cadena más larga) × 100

Ratio = (1 – (1 / 5)) x 100

Ratio = 80% (La diferencia se debe a la implementación de fuzzywuzzy, puede utilizar la distancia de python-Levenshtain o difflib.SequenceMatcher).

Sequence Matcher

SequenceMatcher.ratio() se calcula así:

ratio = 2 . M / |a|+|b|

M es el número de caracteres que coinciden y pueden emparejarse en orden en ambas cadenas. Es decir: cuántos caracteres ‘iguales’ forman la parte común entre los dos textos.

Sequence Matcher

- `a = "Luis"` → |a|=4
- `b = "Luis "` → |b|=5 (el espacio cuenta)
- Los bloques coincidentes emparejan `"Luis"` completo → M=4

Entonces:

$2 \cdot 4 / 4 + 5 = 8/9 = 0.888\ldots \Rightarrow 88.8\%$

Coeficiente de Jaccard

El coeficiente de Jaccard es una métrica utilizada para medir la similitud entre dos conjuntos. Cuando se aplica a cadenas de texto, se basa en la comparación de los conjuntos de n-gramas de cada cadena.

Definición formal

Dado dos conjuntos A y B, el coeficiente de Jaccard se define como:

$J(A,B)= |A \cap B| / |A \cup B|$

donde:

$|A \cap B|$ es el número de elementos en la intersección de los conjuntos A y B.

$|A \cup B|$ es el número de elementos en la unión de los conjuntos A y B.

El resultado varía entre 0 (sin similitud) y 1 (idénticos).

Coeficiente de Jaccard

Aplicación a cadenas de texto

Para aplicar esta métrica a cadenas de caracteres, se deben convertir en conjuntos de n-gramas (subcadenas de longitud n).

Ejemplo

Supongamos que tenemos dos cadenas:

$S1$ = "casa"

S2 = "caso"

Si tomamos bigramas (n=2), los conjuntos serían:

S1 → { "ca", "as", "sa" }

S2 → { "ca", "as", "so" }

Coeficiente de Jaccard

Ahora, calculamos:

- **Intersección**: { "ca", "as" } → tiene 2 elementos.
- **Unión**: { "ca", "as", "sa", "so" } → tiene 4 elementos.

Por lo tanto, el coeficiente de Jaccard es:

$J(S1,S2)=2/4=0.5$

Algoritmos Fonéticos

Son algoritmos diseñados para comparar palabras según su sonido en lugar de su escritura.

Se utilizan en sistemas de búsqueda aproximada, corrección ortográfica y genealogía.

Ayudan a encontrar palabras con pronunciación similar aunque estén escritas de forma diferente.

Algoritmo Soundex

Convierte palabras en un código de 4 caracteres (letra + 3 números).

Reglas:

1. Se conserva la primera letra de la palabra.
2. Se asignan números a las consonantes (Ej: B, F, P, V = 1; C, G, J, K, Q, S, X, Z = 2, etc.).
3. Vocales y algunas consonantes se eliminan.
4. Se rellena o trunca a 4 caracteres.

Ejemplo: "Robert" y "Rupert" tienen el mismo código (R163).

Metaphone

- Mejora Soundex al basarse en reglas fonéticas en inglés.
- Genera claves fonéticas en vez de códigos numéricos.

Metaphone:

- Introducido por Lawrence Philips en 1990.
- Considera reglas de pronunciación en inglés.

Double Metaphone:

- Segunda versión que genera dos claves por palabra (pronunciación primaria y alternativa).
- Mejor precisión con nombres y palabras extranjeras.

Otros Algoritmos Fonéticos

NYSIIS:

- Desarrollado por el estado de Nueva York.
- Mantiene la estructura original de la palabra mejor que Soundex.

Caverphone:

- Diseñado en Nueva Zelanda para nombres propios.
- Se usa para genealogía y bases de datos históricas.

Cologne Phonetic:

- Utilizado en Alemania para comparar nombres fonéticamente.
- Similar a Soundex pero optimizado para el idioma alemán.

Detección de Datos Anómalos (outliers)

¿Qué son los datos anómalos?

Son valores que se desvían significativamente de la tendencia general de los datos. Pueden representar errores o información valiosa.

Métodos para detectar anomalías

Métodos uni-dimensionales

- Histogramas: Identificación de distribuciones irregulares.
- Boxplots
- Rango Intercuartil (IQR): Detecta valores extremos.
- Valores por encima de 3 desviaciones estándar.

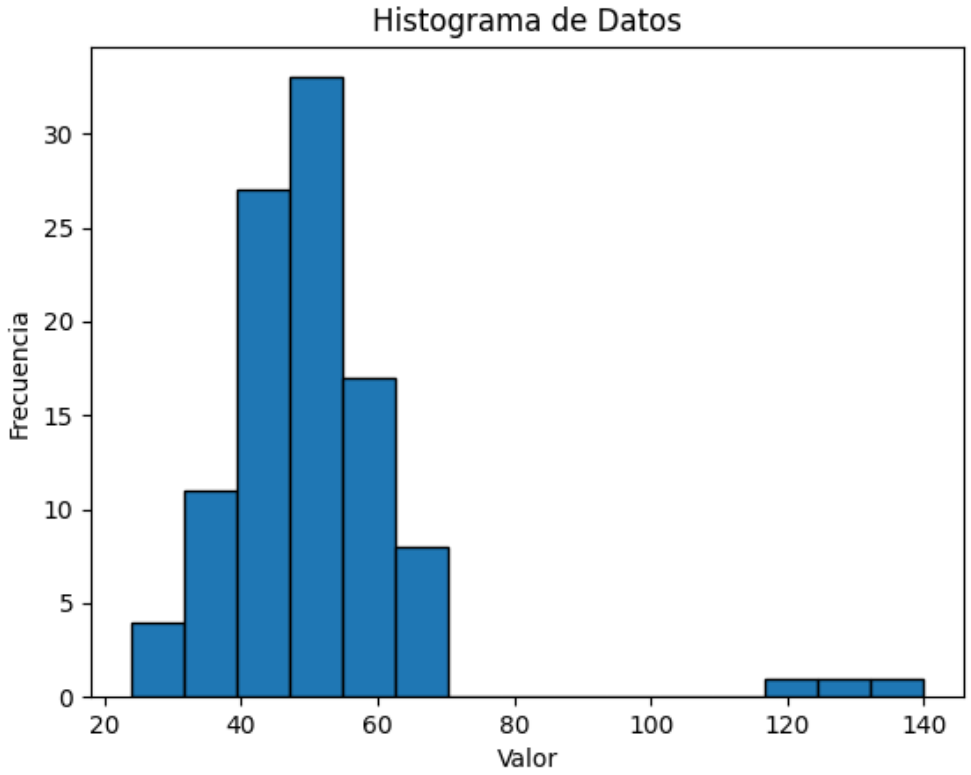
Métodos multidimensionales

- Modelos avanzados: Isolation Forest.

Métodos uni-dimensionales

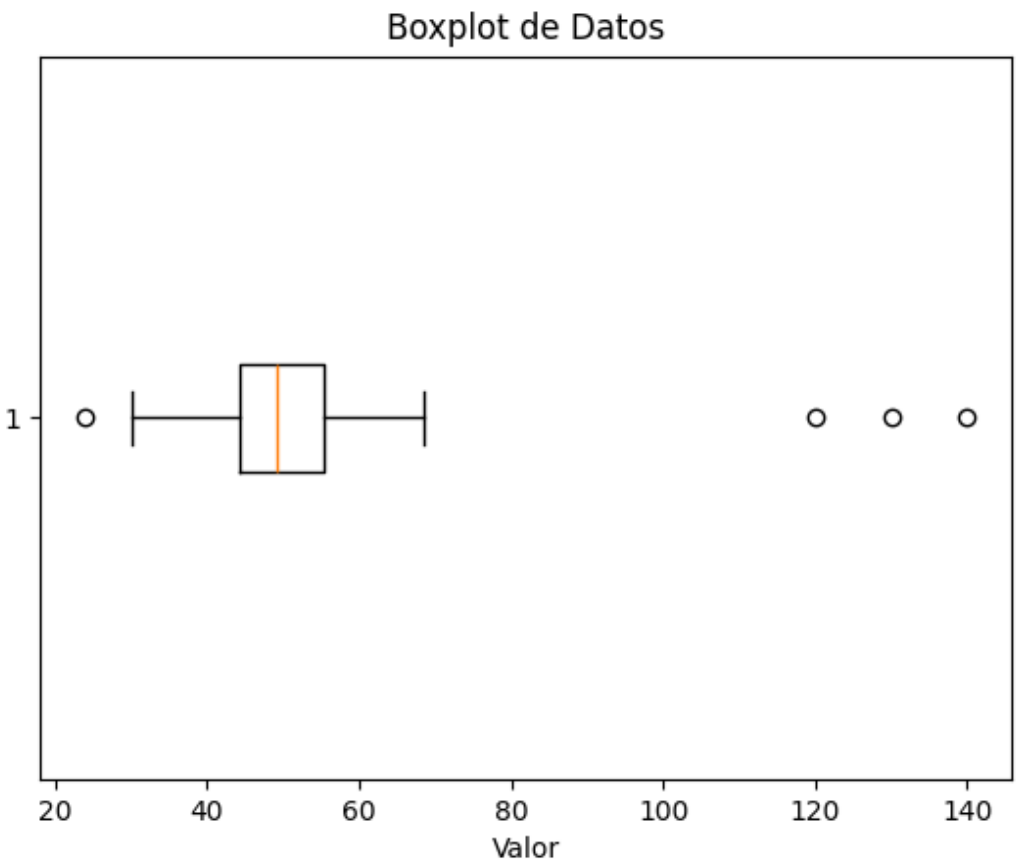
Histogramas

Los histogramas muestran la distribución de los datos y pueden evidenciar valores atípicos en colas largas o picos inesperados.



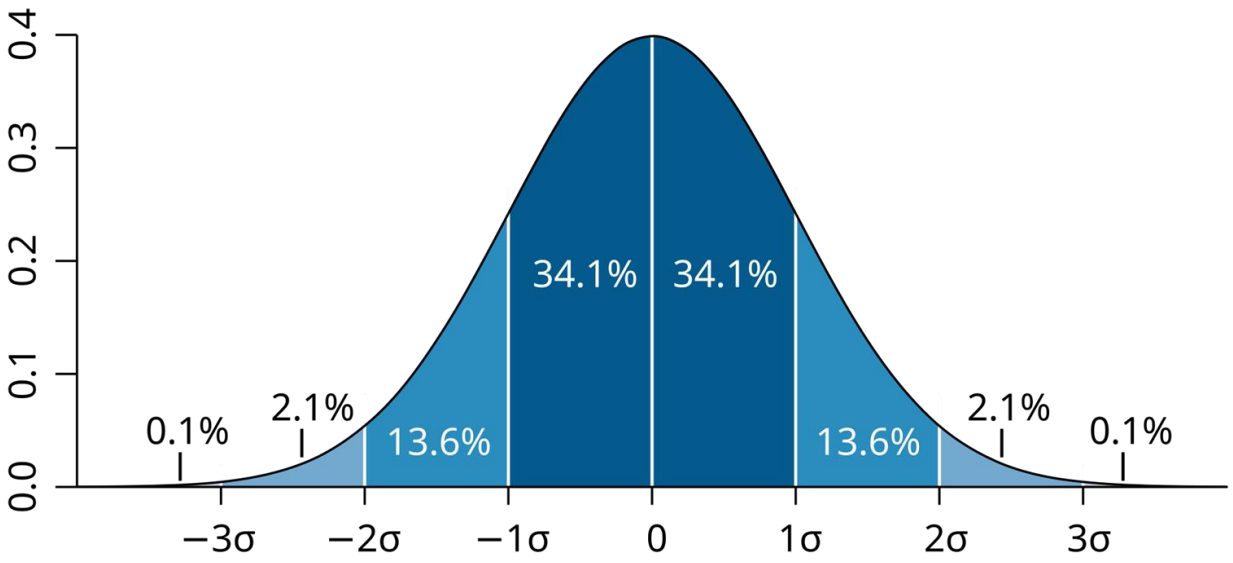
Boxplots

Los boxplots visualizan la dispersión de los datos y marcan valores atípicos fuera de los bigotes.



Valores por encima de 3 Desviaciones Estándar

En una distribución normal, más del 99% de los datos caen dentro de 3 desviaciones estándar de la media. Cualquier dato fuera de este rango puede considerarse un outlier.



Valores por encima de 3 Desviaciones Estándar

Aunque 3 desviaciones estándar es un umbral común, los conjuntos de datos más **pequeños** podrían utilizar **2 desviaciones estándar** para identificar valores atípicos, y los conjuntos de datos más **grandes** podrían utilizar **4 desviaciones estándar** para tener en cuenta una mayor variabilidad.

Rango Intercuartil (IQR)

- El IQR mide la diferencia entre el tercer y el primer cuartil. Se considera outlier cualquier valor por debajo del percentil 25 menos k veces el IQR o por encima del percentil 75 más k veces el IQR .
- El método IQR es preferible en muchos casos porque no presupone la normalidad de la distribución de los datos.
- El método de la desviación estándar supone que los datos siguen una distribución normal, lo que puede no ser siempre el caso. El método IQR es más robusto, ya que utiliza percentiles y puede aplicarse a distribuciones no gaussianas.

Rango Intercuartil (IQR)

- El valor habitual de k es 1,5, pero puede ajustarse a 2 para una detección más estricta de valores atípicos o a 3 para identificar valores atípicos extremos.

Manejo de outliers

Para tratar los valores atípicos detectados, se emplean dos enfoques comunes:

- **Eliminar filas**: Excluir del conjunto de datos filas enteras que contengan valores atípicos.
- **Fijar valores a los límites**: Modificar los valores atípicos para que caigan dentro de un rango aceptable, ya sea fijándolos en el límite inferior o superior.

Métodos multidimensionales

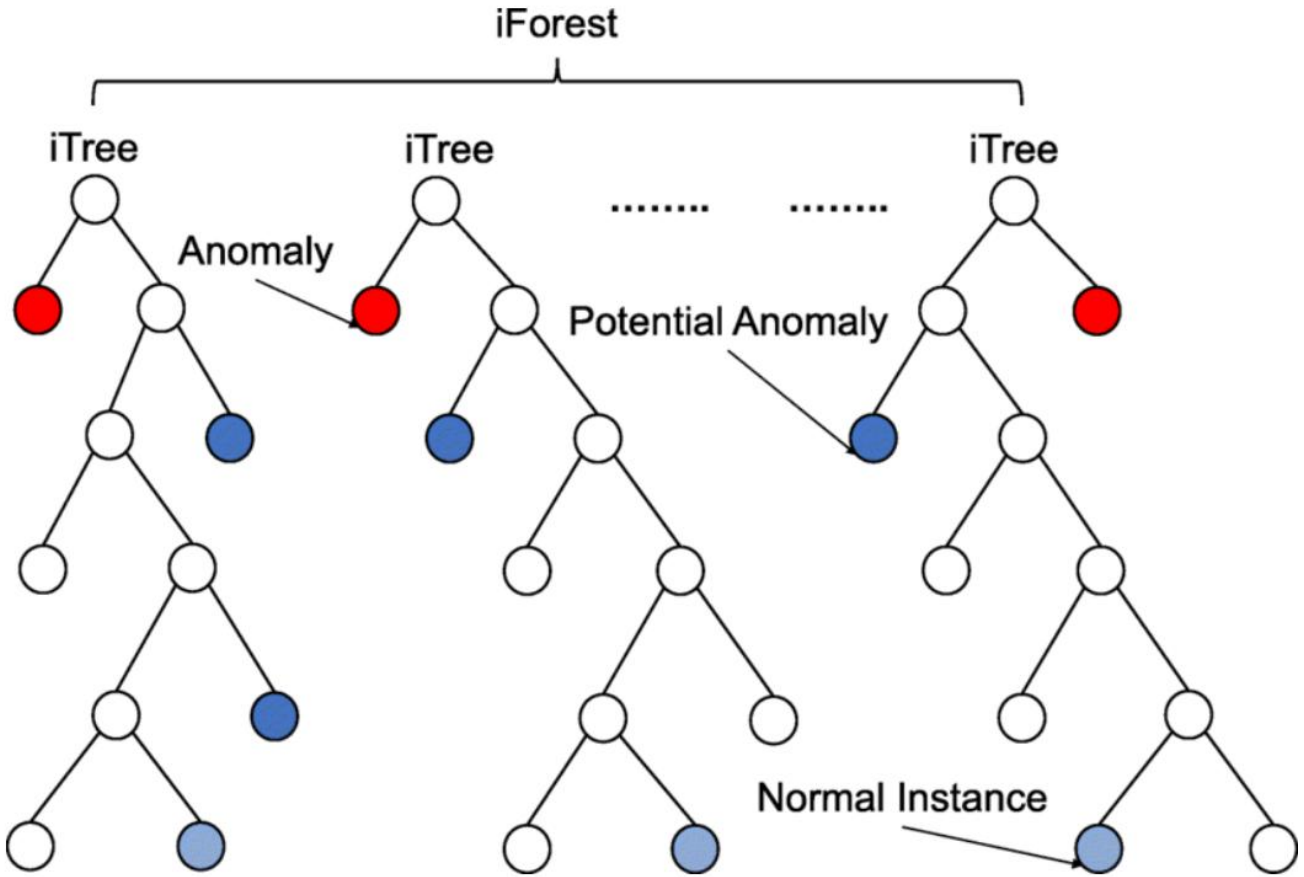
¿Porqué se llaman métodos multidimensionales?

Porque se toma en cuenta más de una dimensión (feature) para decidir si un valor está o no en el rango esperado.

Isolation Forest

Isolation Forest (Bosque de Aislamiento) es un algoritmo basado en árboles de decisión diseñado específicamente para la detección de valores atípicos (outliers). Se basa en la idea de que los valores atípicos son más fáciles de aislar que los valores normales.

¿Cómo funciona Isolation Forest en una dimensión?



¿Cómo funciona Isolation Forest en una dimensión?

En primer lugar, el algoritmo crea un árbol de aislamiento siguiendo los siguientes pasos:

[1] Seleccionar aleatoriamente una submuestra (Sci-kit learn default: 100 instancias/puntos de datos).

[2] Seleccionar un punto para aislar.

[3] Seleccione aleatoriamente una característica (es decir, variable) del conjunto de características X.

[4] Seleccione aleatoriamente un umbral entre el valor mínimo y máximo de la característica x.

[5] Si el punto de datos es menor (mayor) que el umbral, pasa por la rama izquierda del árbol (derecha). En otras palabras, define el nuevo mínimo (máximo) del rango al umbral para la siguiente iteración.

[6] Repita los pasos 3 a 5 hasta aislar el punto o hasta alcanzar un número máximo predefinido de iteraciones.

[7] Registre el número de veces que se repitieron los pasos 3 a 5.

Ejemplo con números (1D)

X = [9.7, 9.8, 9.9, 10.0, 10.1, 10.1, 10.2, 10.3, 10.4, 30.0]

La mayor parte de los puntos son cercanos a 10 y hay un valor extremo:

Idea clave del Isolation Forest (1D):
Cada árbol elige **umbrales aleatorios** (t) dentro del rango actual ([min, max]) y parte el conjunto con reglas del tipo:

- izquierda: ($x \leq t$)
- derecha: ($x > t$)

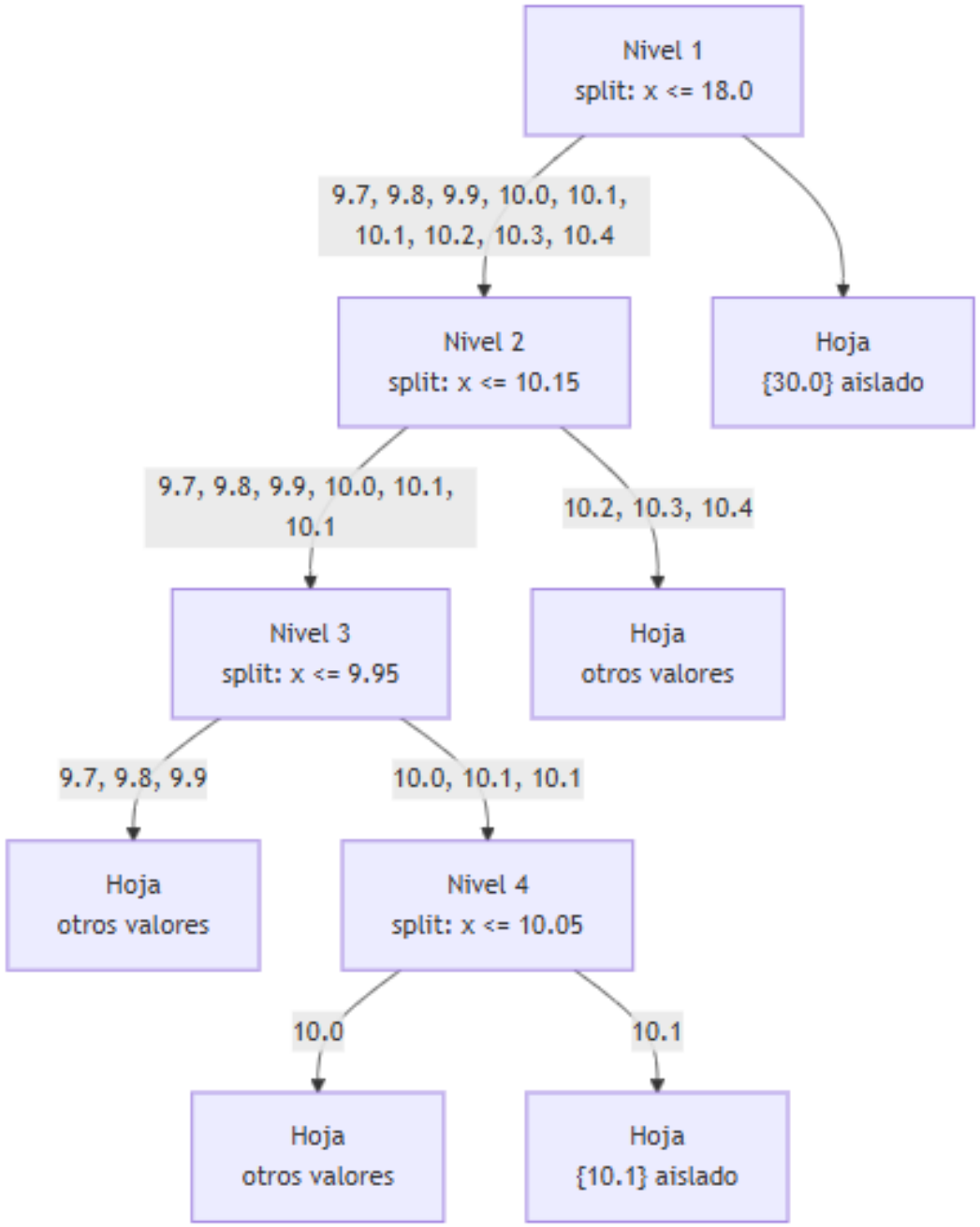
Los **outliers** tienden a quedar aislados en **pocos splits** (profundidad pequeña).

Ejemplo con números (1D)

Punto 30
En [9.7,18] split t=18 → 30 cae a la derecha y queda aislado

Punto 10.1
En [9.7,30] split t=18 → 10.1 cae a la izquierda
En [9.7,10.4] split t=10.15 → 10.1 cae a la izquierda
En [9.7,10.1] split t=9.95 → 10.1 cae a la derecha
En [10.0,10.1] split t=10.05 → 10.1 cae a la derecha y queda aislado

Resumen Árbol 1 (umbrales):
(18.0, 10.15, 9.95, 10.05)
Para **10.1**: (h(10.1)=4)



Ejemplo con números (1D)

Suponiendo que se realiza el mismo proceso 3 veces y que los resultados obtenidos se utilizan para calcular el promedio de profundidades (path length)

E[h(30.0)] = (1+1+1)/3 = 1
E[h(10.1)] = (4+6+7)/3 aprox. 5.67

Score de anomalía

Para (n=10), (c(10) ≈ 3.86).

s(30)=2^{-1/3.86} ≈ 0.836

s(10.1)=2^{-5.67/3.86} ≈ 0.361

$$s(x,n) = 2^{-\frac{E(h(x))}{c(n)}}$$

where E[h(x)] is the average number of successful iterations for value x

c(n) es una **constante de normalización** en Isolation Forest: representa la **longitud de camino promedio** (average path length) esperada para aislar un punto en un árbol construido por particiones aleatorias. Para n=10 c(10) ≈ 3.86

Constante de iteraciones para diferentes n’s

$c(10) \approx 3.86$

$c(100) \approx 8.36$

$c(1000) \approx 12.97$

Intuición: con más puntos, en promedio necesitas **más cortes** para aislar uno, por eso sube c(n).

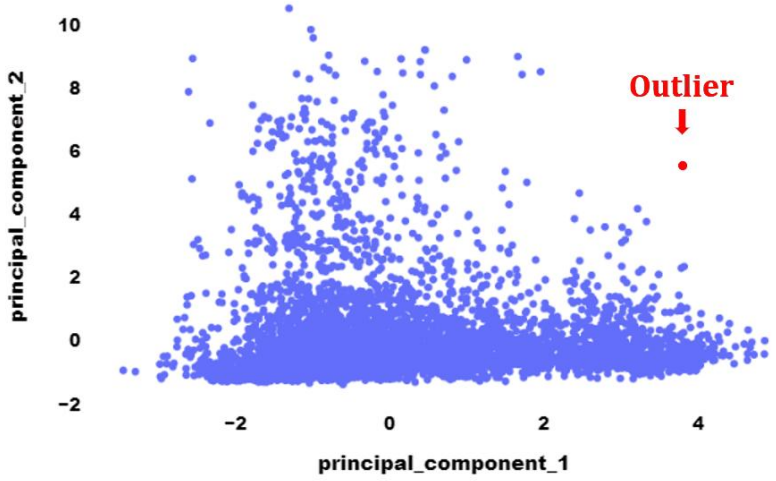
¿Y el “umbral” para decir outlier / no outlier?

Hay dos formas típicas:

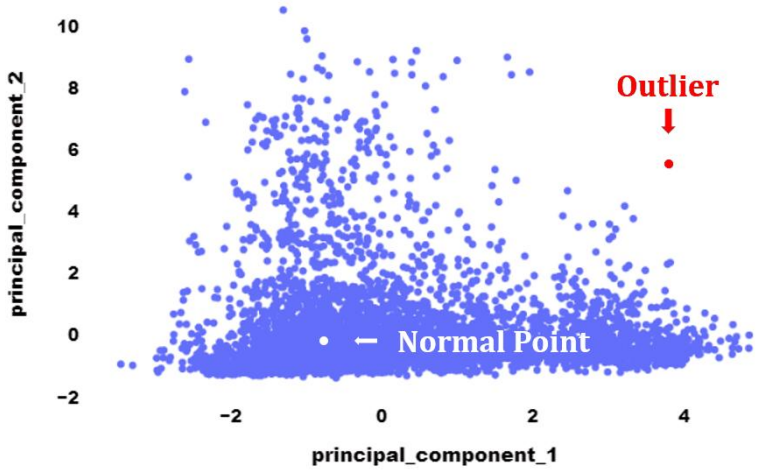
1. **Por contaminación (contamination = 0.10)**
   Si dices “espero ~10% outliers”, con 10 datos marcas como outlier al **top 1** con mayor score → aquí sería **30.0**.
2. **Por percentil / corte en score**
   Ejemplo didáctico: “marco outliers si (s(x) > 0.6)”
   - 30.0 (0.836) ⇒ outlier
   - 10.1 (0.361) ⇒ normal

¿Cómo funciona Isolation Forest en dos dimensiones?

Isolation Forest randomly splits the sample until a point is "isolated."



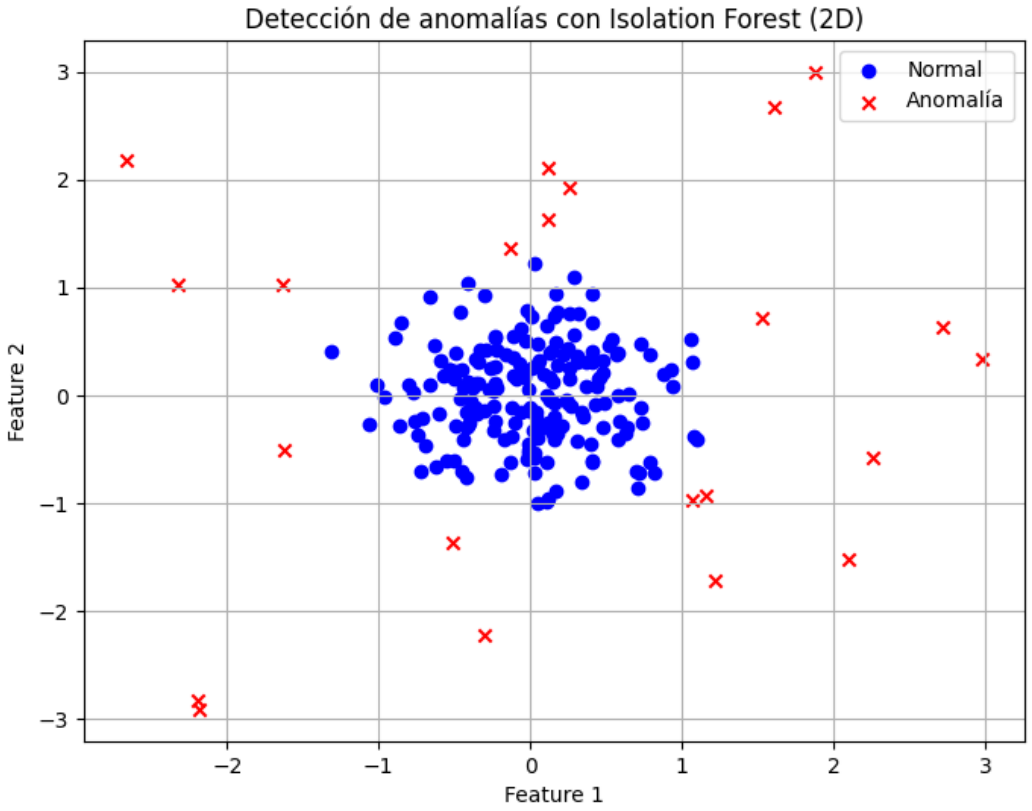
Unlike the red point, the white point will take a lot of splits to isolate it.



https://medium.com/@y.s.yoon/isolation-forest-anomaly-detection-identify-outliers-101123a9ff63

Ejemplo en código

Isolation Forest



Detección y Medición de Datos Incompletos (Completitud)

Detección y Medición de Datos Incompletos (Completitud)

¿Qué es la completitud de datos?

La completitud mide la cantidad de datos faltantes en un conjunto de datos. Un dataset incompleto puede afectar la validez de los análisis y decisiones.

Causas de datos faltantes

- Errores en la captura: Omisión de información en encuestas o formularios.
- Problemas de integración: Incompatibilidad de fuentes de datos.
- Supresión intencional: Falta de información por políticas de privacidad.

Métodos para detectar valores faltantes

1) Conteo de valores nulos (isnull() en pandas)
2) Visualización de patrones de valores faltantes (missingno)
3) Generar un informe sobre el porcentaje de registros con datos faltantes.
4) Crear un informe detallado sobre la calidad de los datos

Conteo de valores nulos

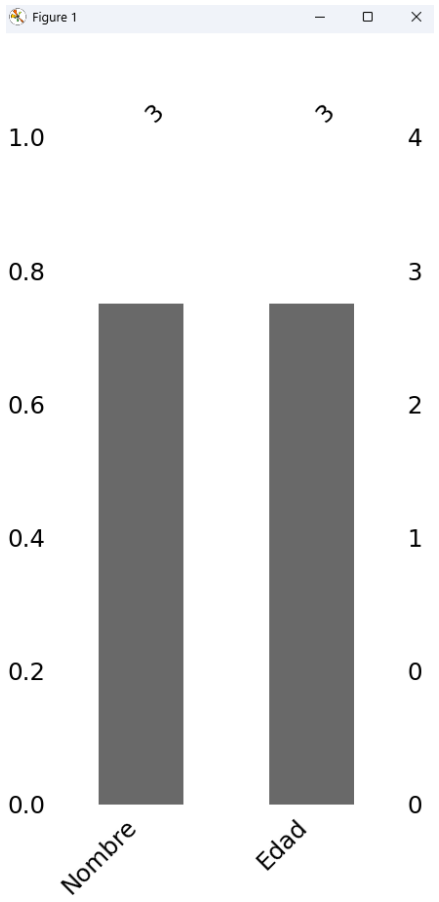
```
import pandas as pd
import numpy as np

# Crear un DataFrame con valores faltantes
data = {'Nombre': ['Ana', 'Luis', 'Maria', np.nan], 'Edad': [25, np.nan, 30, 22]}
df = pd.DataFrame(data)

# Contar valores faltantes por columna
print(df.isnull())
print(df.isnull().sum())
```

Visualización de patrones de valores faltantes

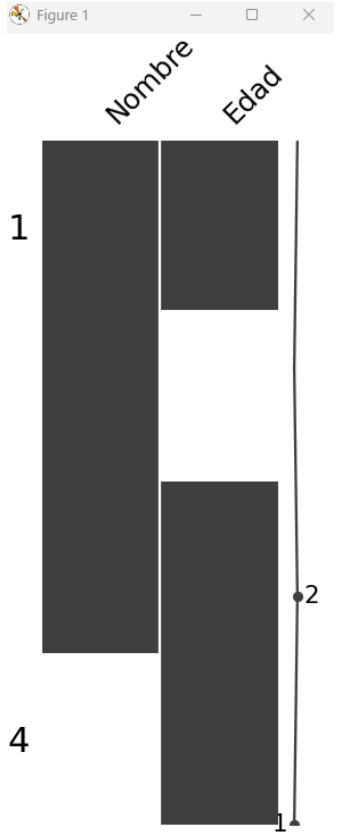
```
import pandas as pd

import numpy as np

import missingno as msno

import matplotlib.pyplot as plt

# Crear un DataFrame con valores faltantes

data = {'Nombre': ['Ana', 'Luis', 'Maria', np.nan], 'Edad': [25, np.nan, 30, 22]}

df = pd.DataFrame(data)

# Visualizar valores faltantes

msno.bar(df)

plt.show()

msno.matrix(df)

plt.show()
```

Visualización de patrones de valores faltantes

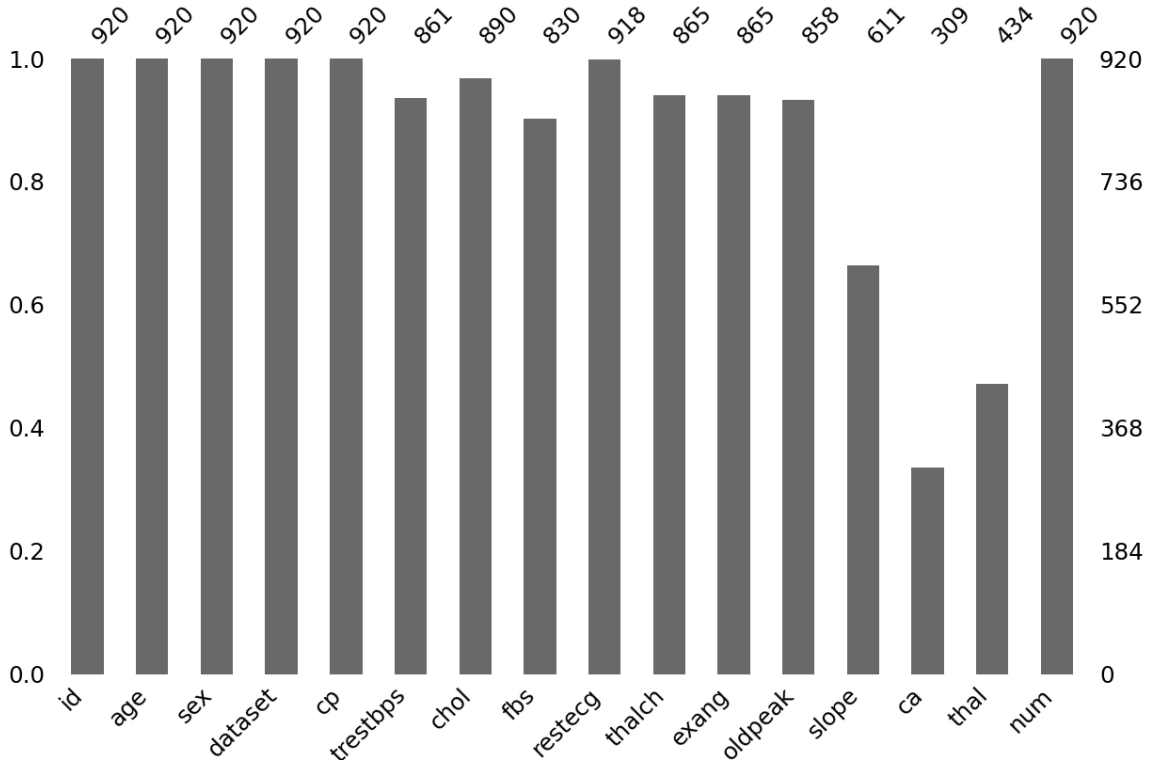
msno.bar(df)



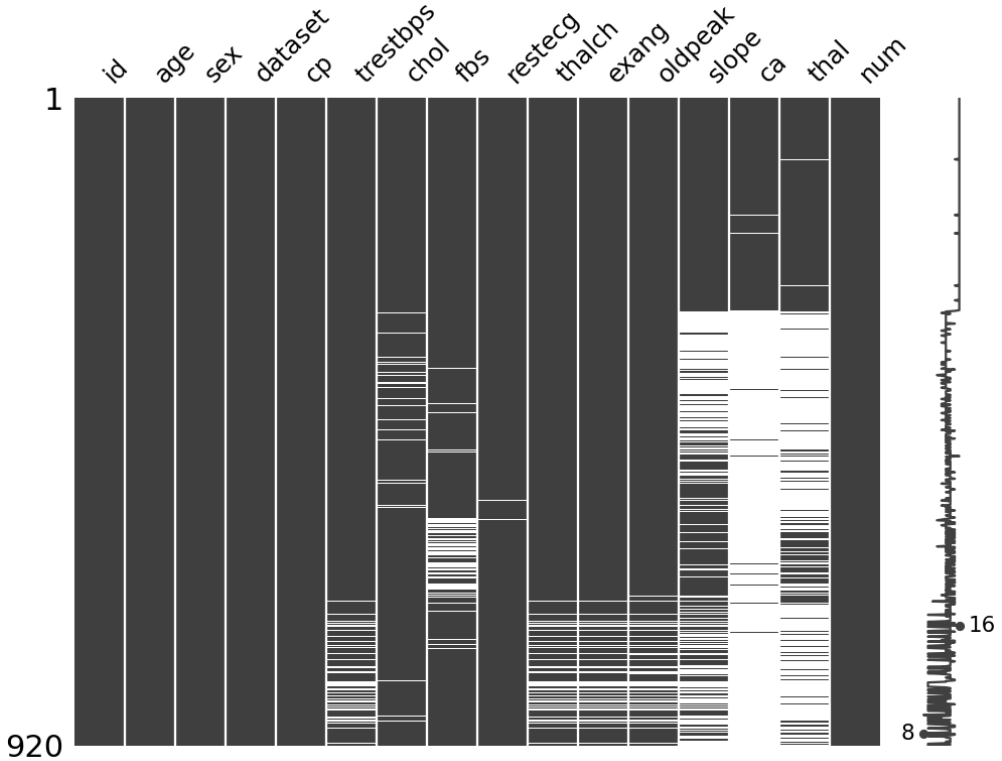
msno.matrix(df)



Aplicando missingno a un dataset de enfermedades cardiacas (heart_disease_uci)



msno.bar(df)



msno.matrix(df)

Métodos para detectar valores faltantes

1) **Generar un informe sobre el porcentaje de registros con datos que faltan.**
   a) Si el porcentaje de datos que faltan es bajo, considere la posibilidad de descartar esos registros.

1) **Crear un informe detallado**
   a) Para porcentajes más altos de datos faltantes, se puede generar un informe detallado basado en el número de celdas que faltan en cada registro.
   b) Ordenar este informe para identificar los registros con un número significativo de celdas omitidas.
   c) Descartar los registros con un número significativo de celdas omitidas para mantener la calidad de los datos y evitar análisis engañosos.

Elaboración de Reporte de Diagnóstico de Calidad de los Datos

Elaboración de Reporte de Diagnóstico de Calidad de los Datos

Elementos clave del reporte

1. Resumen del dataset
2. Análisis de duplicados
3. Identificación de valores faltantes
4. Detección de anomalías
5. Visualizaciones
6. Recomendaciones

Cómo leer un histograma en 10 segundos (regla práctica)

Eje X: valores (o rangos de valores).

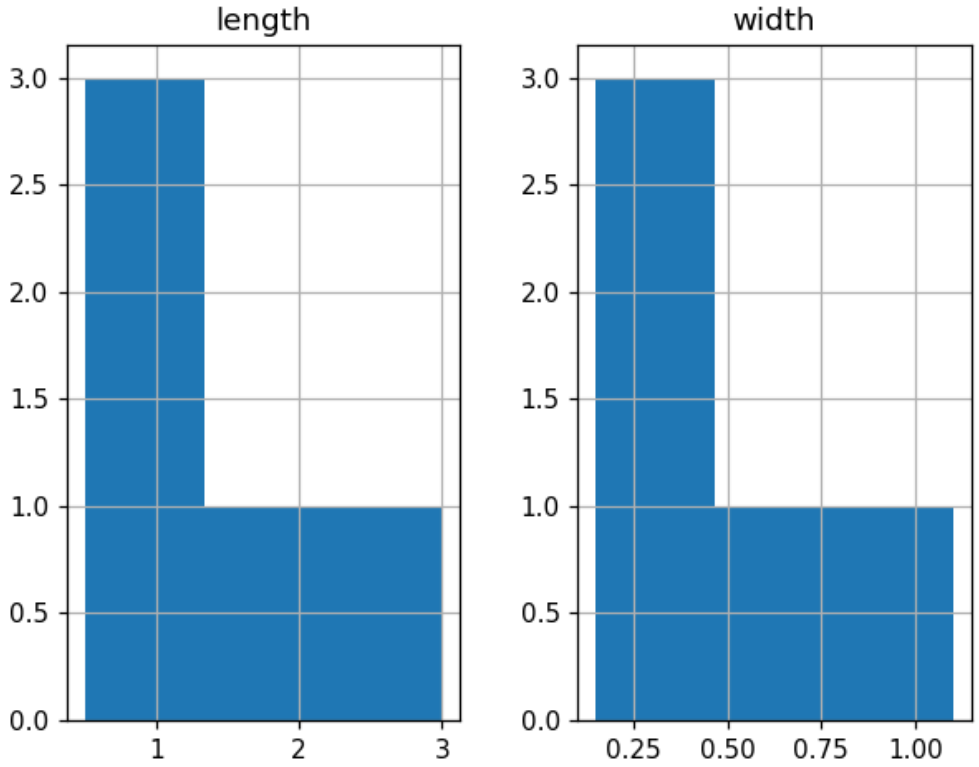
Eje Y: cuántas observaciones caen en cada rango.

Si ves una barra “solitaria” en los extremos, suele indicar un **valor atípico** o al menos un valor mucho mayor/menor que el resto.

Cómo leer un histograma en 10 segundos (regla práctica)

```
import pandas as pd

import matplotlib.pyplot as plt


data = {

     "length": [1.5, 0.5, 1.2, 0.9, 3],

     "width": [0.7, 0.2, 0.15, 0.2, 1.1],

}

index = ["pig", "rabbit", "duck", "chicken", "horse"]

df = pd.DataFrame(data, index=index)

hist = df.hist(bins=3)

plt.show()
```

	length	width
pig	1.5	0.70
rabbit	0.5	0.20
duck	1.2	0.15
chicken	0.9	0.20
horse	3.0	1.10



¿Cuáles son los bins exactos con bins=3?

Por defecto, df.hist(bins=3) usa **bins de ancho uniforme** (vía matplotlib/numpy)

1) Para la columna **length**

Valores: [1.5, 0.5, 1.2, 0.9, 3.0]

mínimo = **0.5**

máximo = **3.0**

rango = 2.5

bin width = 2.5 / 3 = **0.833333...**

Edges (límites) exactos:

- 0.5
- 1.3333333333
- 2.166666666
- 3.0

Bins:

- [0.5 , 1.33)
- [1.33,2.166)
- [2.166 , 3.0]

Generación de un reporte manual

```
# Visualizar las primeras filas del dataset
print("Primeras filas del dataset:")
print(df.head())
```

```
Primeras filas del dataset:
   id  age     sex    dataset               cp  ...  oldpeak        slope   ca              thal  num
0   1   63    Male  Cleveland   typical angina  ...      2.3  downsloping  0.0       fixed defect    0
1   2   67    Male  Cleveland     asymptomatic  ...      1.5         flat  3.0             normal    2
2   3   67    Male  Cleveland     asymptomatic  ...      2.6         flat  2.0  reversable defect    1
3   4   37    Male  Cleveland      non-anginal  ...      3.5  downsloping  0.0             normal    0
4   5   41  Female  Cleveland  atypical angina  ...      1.4    upsloping  0.0             normal    0

[5 rows x 16 columns]
```

Reporte manual

```python
# Información general del dataset
print("\nInformación del dataset:")
df.info()
```

```
Información del dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 920 entries, 0 to 919
Data columns (total 16 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   id        920 non-null    int64
 1   age       920 non-null    int64
 2   sex       920 non-null    object
 3   dataset   920 non-null    object
 4   cp        920 non-null    object
 5   trestbps  861 non-null    float64
 6   chol      890 non-null    float64
 7   fbs       830 non-null    object
 8   restecg   918 non-null    object
 9   thalch    865 non-null    float64
 10  exang     865 non-null    object
 11  oldpeak   858 non-null    float64
 12  slope     611 non-null    object
 13  ca        309 non-null    float64
 14  thal      434 non-null    object
 15  num       920 non-null    int64
dtypes: float64(5), int64(3), object(8)
memory usage: 115.1+ KB
```

Reporte manual

```
Estadísticas descriptivas:
                id         age    trestbps  ...     oldpeak          ca         num
count  920.000000  920.000000  861.000000  ...  858.000000  309.000000  920.000000
mean   460.500000   53.510870  132.132404  ...    0.878788    0.676375    0.995652
std    265.725422    9.424685   19.066070  ...    1.091226    0.935653    1.142693
min      1.000000   28.000000    0.000000  ...   -2.600000    0.000000    0.000000
25%    230.750000   47.000000  120.000000  ...    0.000000    0.000000    0.000000
50%    460.500000   54.000000  130.000000  ...    0.500000    0.000000    1.000000
75%    690.250000   60.000000  140.000000  ...    1.500000    1.000000    2.000000
max    920.000000   77.000000  200.000000  ...    6.200000    3.000000    4.000000

[8 rows x 8 columns]
```

```
# Estadísticas descriptivas
print("\nEstadísticas descriptivas:")
print(df.describe())
```

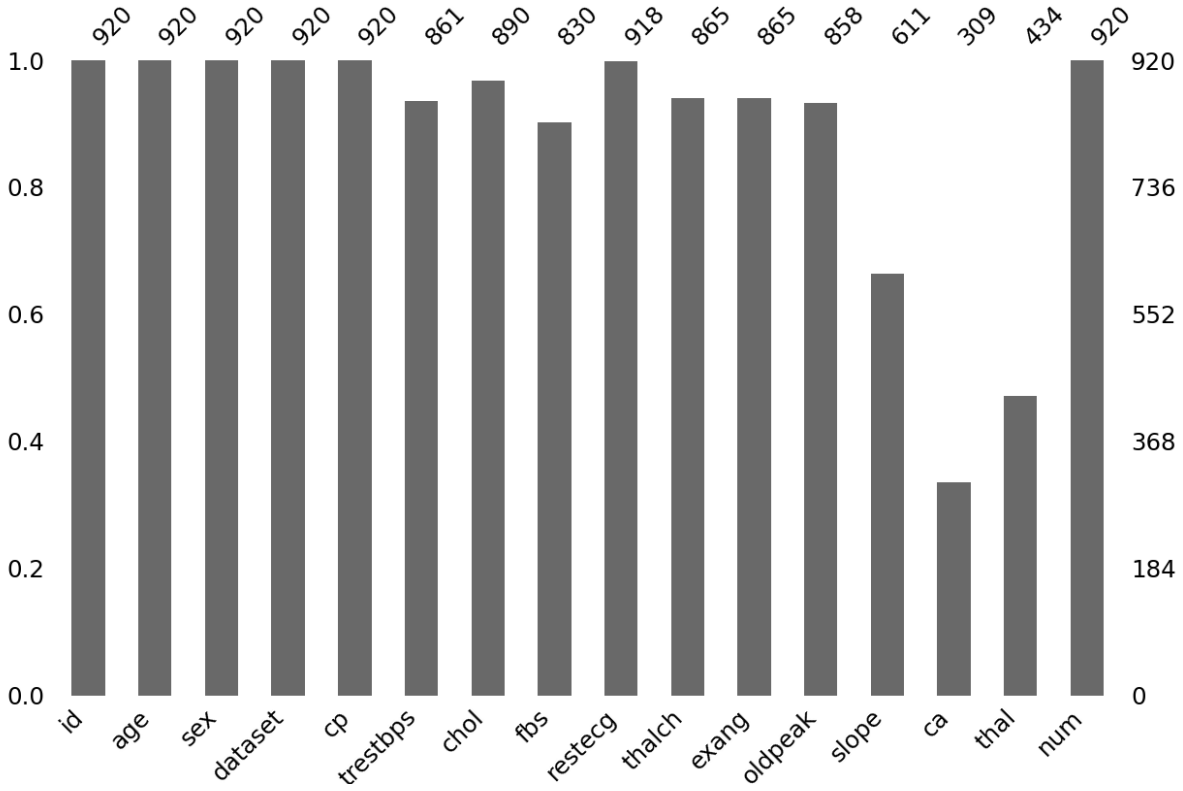
Reporte manual

```
# Detección de valores faltantes
print("\nValores faltantes por columna:")
print(df.isnull().sum())
```

```
Valores faltantes por columna:
id            0
age           0
sex           0
dataset       0
cp            0
trestbps     59
chol         30
fbs          90
restecg       2
thalch       55
exang        55
oldpeak      62
slope       309
ca          611
thal        486
num           0
dtype: int64
```

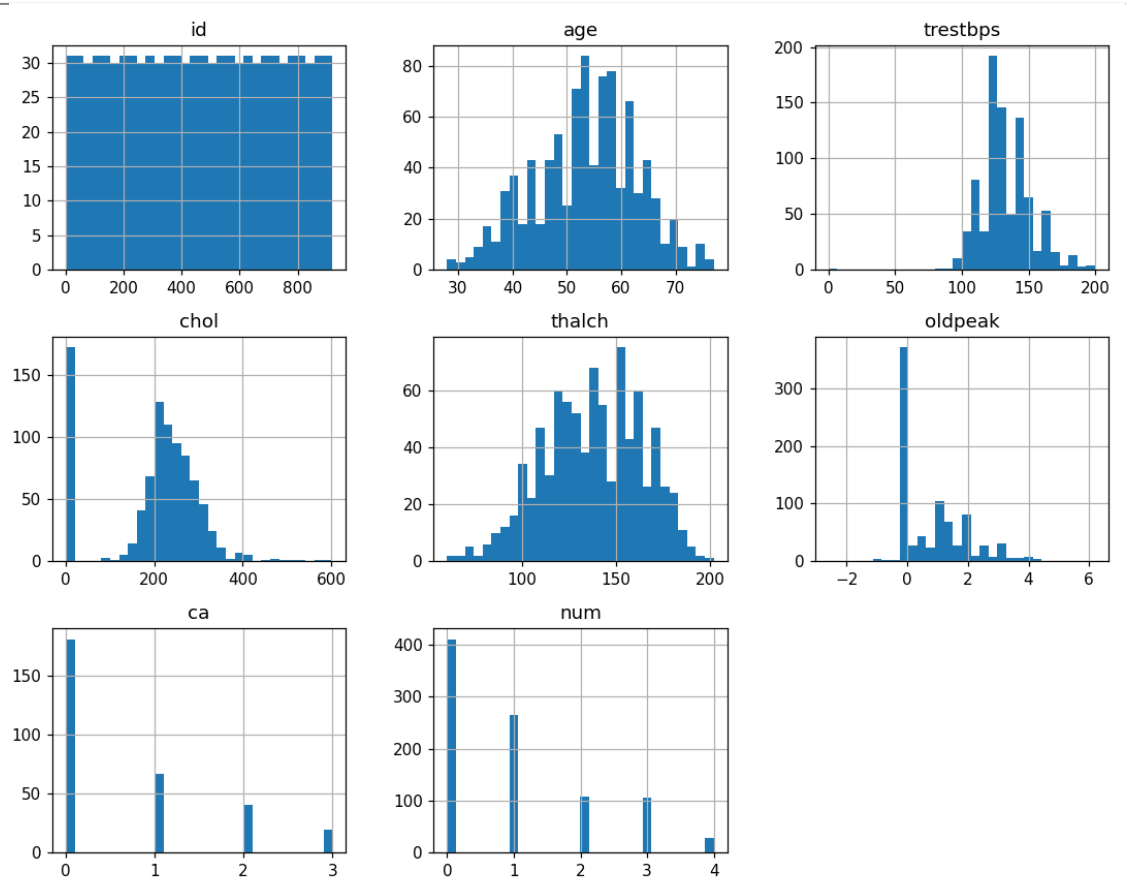
Reporte manual

```
# Visualización de valores faltantes
msno.bar(df)
plt.show()
```

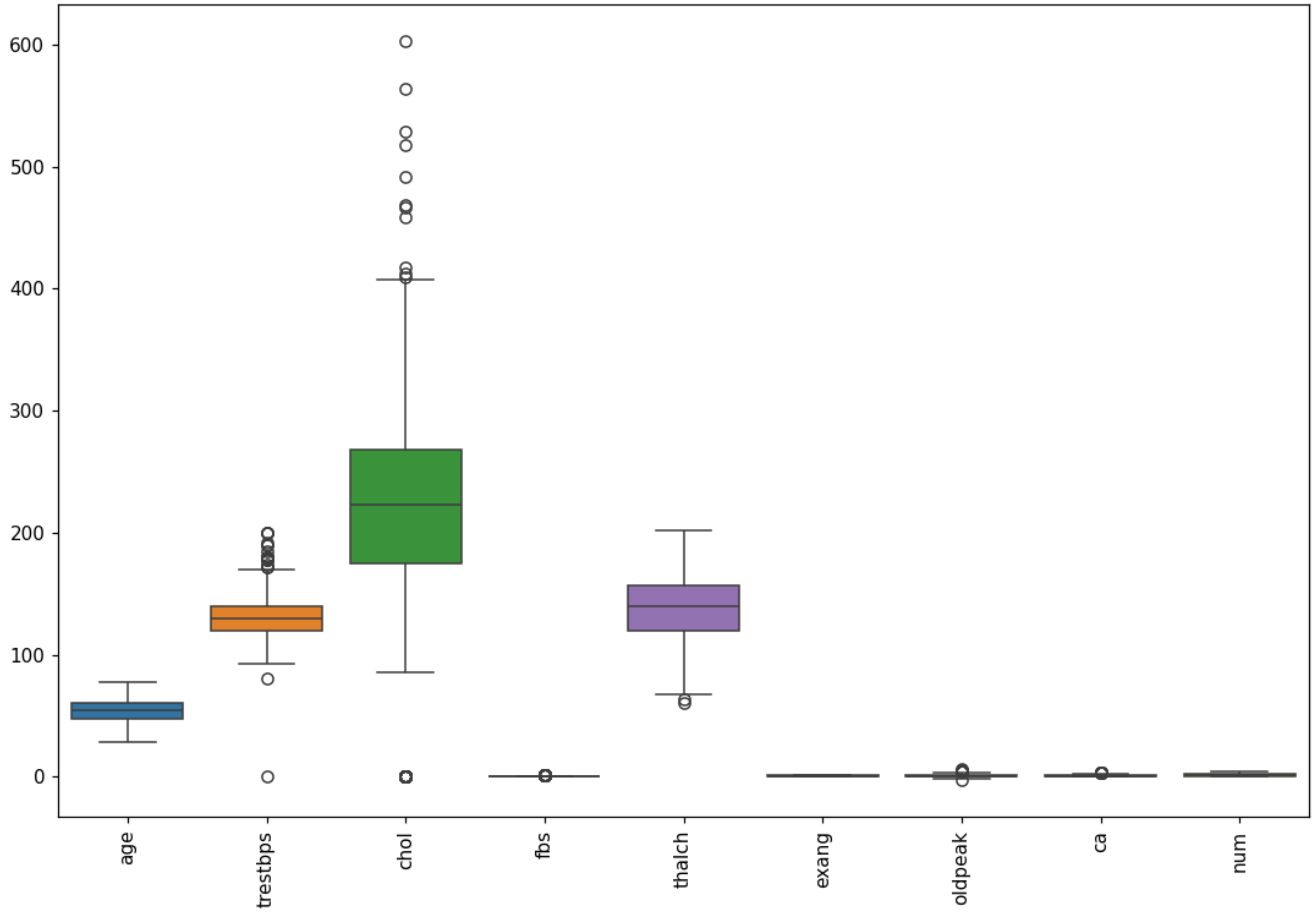


Histogramas de la base de datos heart_disease_uci

```
# Distribución de las variables numéricas
df.hist(figsize=(12, 10), bins=30)
plt.show()
```



Boxplots de la base de datos heart_disease_uci




```
# Boxplots para detectar valores atípicos
plt.figure(figsize=(12, 6))
sns.boxplot(data=df[['age', 'sex', 'dataset',
'cp', 'trestbps', 'chol', 'fbs',
       'restecg', 'thalch', 'exang',
'oldpeak', 'slope', 'ca', 'thal', 'num']])
plt.show()
```

Reporte manual

```
Número de registros duplicados: 0
Index(['id', 'age', 'sex', 'dataset', 'cp', 'trestbps', 'chol', 'fbs',
       'restecg', 'thalch', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'],
      dtype='object')
```

```
# Detección de duplicados
duplicados = df.duplicated().sum()
print(f"\nNúmero de registros duplicados: {duplicados}")


print(df.columns)
```

Generación automática con ‘pandas-profiling’

```
import pandas as pd

from ydata_profiling import ProfileReport
# Cargar el dataset
file_path = "heart_disease_uci.csv"
df = pd.read_csv(file_path)

# Generación de un reporte automático de perfilado de datos
profile = ProfileReport(df, title="Reporte de Perfilado de Datos - Heart Disease UCI")
profile.to_file("heart_disease_profiling_report.html")
```

Métodos de imputación de datos

- ✓Imputación con valores estadísticos: Media, mediana, moda.
- ✓Imputación con modelos predictivos: Iterative Imputer, KNN Imputer.

Métodos de imputación de datos

La **imputación** es simplemente el proceso de rellenar los "huecos" en los datos con estimaciones estadísticas. Esto nos permite conservar los registros incompletos en lugar de eliminarlos, lo cual podría significar una pérdida valiosa de información.

Las técnicas más rápidas y comunes utilizan las medidas de tendencia central. A continuación, desglosaremos la imputación por la media, la mediana y la moda.

Imputación con la Media (Promedio)

La media es el promedio aritmético de todos los valores válidos en una columna. Se calcula sumando todos los números y dividiéndolos por la cantidad de observaciones. Matemáticamente se representa así:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

- **¿Cuándo usarla?** Es ideal para variables numéricas continuas que tienen una distribución normal o simétrica (forma de campana) y **no tienen valores atípicos** (outliers extremos).
- **El peligro:** La media es extremadamente sensible a los valores atípicos. Un solo número muy grande o muy pequeño puede arrastrar el promedio y hacer que tu imputación sea irreal.

Imputación con la Media (Promedio)

Ejemplo Práctico:

Imagina que mides las estaturas (en cm) de 5 jugadores de basketball, pero se borró el dato de uno:

[198, 201, 195, NaN, 200]

Sumamos los datos existentes: 198 + 201 + 195 + 200 = 794

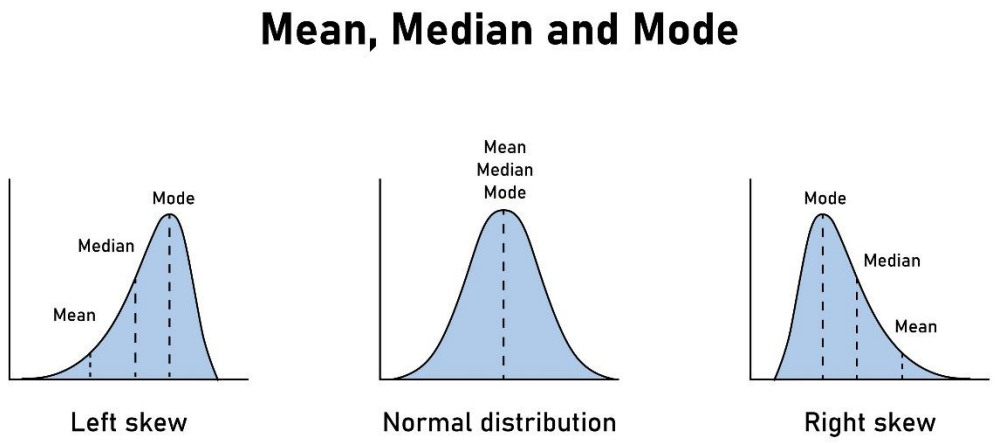
Dividimos entre la cantidad de datos no nulos (4): 794 / 4 = 198.5

Resultado: Reemplazamos el NaN por 198.5. Como los datos se parecen entre sí y las estaturas comúnmente tienen una distribución normal, la media es un excelente representante del grupo.

Imputación con la Mediana

La mediana es el valor central exacto de tus datos una vez que los ordenas de menor a mayor. Si hay una cantidad par de números, es el promedio de los dos centrales.

¿Cuándo usarla? Es la mejor opción para variables numéricas cuando tus datos están **sesgados** o tienen **valores atípicos** muy fuertes. La mediana es "robusta", lo que significa que no se deja engañar por extremos irreales.



Imputación con la Mediana

Ejemplo Práctico:

Supongamos que analizas los salarios mensuales (en dólares) de una pequeña startup de 5 personas. Tienes los sueldos de los analistas y el del CEO, pero te falta el sueldo de un nuevo analista:

[3000, 3200, 3100, NaN, 50000] *(El 50,000 es el CEO)*

Si usamos la **media**, el promedio sería 14,825. Imputar el dato faltante del nuevo analista con 14,825 distorsionaría totalmente la realidad.

Si usamos la **mediana**, primero ordenamos los valores válidos: 3000, 3100, 3200, 50000. El punto medio está entre 3100 y 3200.

Calculamos: (3100 + 3200) / 2 = 3150.

Resultado: Reemplazamos el NaN con 3150. Este valor es mucho más representativo de lo que ganaría un analista promedio en esa empresa.

Imputación con la Moda

La moda es simplemente el valor o categoría que aparece con mayor frecuencia en tu conjunto de datos.

¿Cuándo usarla? Es el método principal (y a menudo el único de estos tres) que se utiliza para **variables categóricas** (texto, etiquetas, categorías) o variables numéricas estrictamente discretas donde promediar no tiene sentido lógico (por ejemplo, el número de puertas de un coche).

Imputación con la Moda

Ejemplo Práctico:

Imagina que tienes una base de datos de las tallas de camisetas que compran tus clientes, y falta un registro:

["M", "L", "M", "S", "M", NaN, "L", "XL", "M"]

Contamos las frecuencias:

- "S" = 1
- "M" = 4
- "L" = 2
- "XL" = 1

El valor más repetido es "M".

Resultado: Rellenamos el valor faltante NaN con "M", asumiendo que el cliente con el dato faltante tiene la talla más común.